CS 6650 — Building Scalable Distributed Systems

Northeastern University · Final Project Report · April 2026

TEAM MEMBERS

Anand Mohan Singh	Infrastructure (Terraform, 9 modules, Makefile)
Vaibhav Thalanki	API development (Go Gin, DynamoDB schema, concurrency logic)
Pranav Viswanathan	Printer worker, experiment scripts, CloudWatch observability

REPOSITORY

https://github.com/anandms101/CampusCloud-Print-Queue

DELIVERABLE INDEX

§1	Video Presentation	5 marks
§2	Code and Repository	5 marks
§3	Project Management	5 marks
§4	Experiments Report	10 marks
§5	Community Contributions	5 marks

Contents

1 Video Presentation [5 marks] 3
- 1.1 Segment 1 — Anand Mohan Singh (2–3 minutes) . . . 3
- 1.2 Segment 2 — Vaibhav Thalanki (2–3 minutes) . . . 3
- 1.3 Segment 3 — Pranav Viswanathan (2–3 minutes) . . . 3

2 Code and Repository [5 marks] 3
- 2.1 Primary Repository . . . 3
- 2.2 README Overview . . . 4
- 2.3 Repository Structure . . . 4
- 2.4 CI Pipeline . . . 4
- 2.5 Testing Coverage . . . 4

3 Project Management [5 marks] 5
- 3.1 Work Breakdown by Team Member . . . 5
- 3.2 Project Timeline . . . 5
- 3.3 Design Evolution and Problems Encountered . . . 5
  - 3.3.1 Initial Design vs. Final State . . . 6
  - 3.3.2 Problems Encountered . . . 6
- 3.4 Cost Model . . . 6

4 Experiments [10 marks] 7
- 4.1 System Overview . . . 7
- 4.2 Experiment 1: API Load Test . . . 7
  - 4.2.1 Purpose and Tradeoff Explored . . . 7
  - 4.2.2 Methodology . . . 7
  - 4.2.3 Results . . . 8
  - 4.2.4 Analysis . . . 9
- 4.3 Experiment 2: DynamoDB Contention Test . . . 9
  - 4.3.1 Purpose and Tradeoff Explored . . . 9
  - 4.3.2 Methodology . . . 9
  - 4.3.3 Results . . . 10
  - 4.3.4 Analysis . . . 11
- 4.4 Experiment 3: Printer Saturation Test . . . 11
  - 4.4.1 Purpose and Tradeoff Explored . . . 11
  - 4.4.2 Methodology . . . 11
  - 4.4.3 Results . . . 12
  - 4.4.4 Analysis . . . 13
- 4.5 Experiment 4: Fault Injection — Printer Worker Kill and Recovery . . . 13
  - 4.5.1 Purpose and Tradeoff Explored . . . 13
  - 4.5.2 Methodology . . . 14
  - 4.5.3 Results . . . 14
  - 4.5.4 Analysis . . . 15

5 Community Contributions [5 marks] 15
- 5.1 Piazza Posting . . . 15

5.2 Related Projects 15
5.2.1 Expected Similarity Criteria 15
5.2.2 Comparison Template (to be completed post-posting) 16

Appendix: Related Work **16**

1. Video Presentation [5 marks]

The submission includes one screen-capture video in which each team member presents the portion of the project they personally built. No AI voice-overs are used; all narration is performed live by the respective team member. The video is segmented as follows.

1.1 Segment 1 — Anand Mohan Singh (2–3 minutes)

Anand walks through the infrastructure layer: the nine Terraform modules (`networking`, `ecr`, `iam`, `dynamodb`, `s3`, `sqs`, `alb`, `ecs`, `cloudwatch`), the use of `for_each` over the `printer_names` variable to instantiate per-printer SQS queues and ECS services, and the Makefile automation (`make deploy-fresh`, `make teardown`, `make queue-depth`, etc.). He demonstrates a live `terraform plan` output and shows the CloudWatch dashboard with all six panels active.

1.2 Segment 2 — Vaibhav Thalanki (2–3 minutes)

Vaibhav presents the Go Gin API service: the seven endpoint handlers, the DynamoDB conditional expression logic used for every state transition, the GSI schema design (`userId-createdAt-index`), middleware layering (request ID, structured logging, timeout, rate limiter), the circuit breaker configuration via `sony/gobreaker/v2`, and the bulkhead semaphore around the upload handler. He also demonstrates the FastAPI variant and explains the Bulkhead isolation applied to the `POST /jobs` endpoint in the Go Gin implementation.

1.3 Segment 3 — Pranav Viswanathan (2–3 minutes)

Pranav covers the Python printer worker: the SQS long-polling loop, the idempotency state machine in `processor.py` including the crash recovery branch (`status=PROCESSING` and `receiveCount > 1`), the visibility-extension heartbeat, and the compensating logic for S3 download failures. He then walks through all four experiment scripts (`locustfile.py`, `contention_test.py`, `saturation_test.py`, `fault_injection_test.py`) and the CloudWatch log group and dashboard configuration.

Video link: https://github.com/anandms101/CampusCloud-Print-Queue (linked from README)

2. Code and Repository [5 marks]

2.1 Primary Repository

https://github.com/anandms101/CampusCloud-Print-Queue

The repository was developed collaboratively with commits from all three team members. Commit history, branch activity, and issue tracking reflect the progression of the project from initial architecture through final experiment execution.

2.2 README Overview

The `README.md` introduces the project motivation (release-at-device printing), explains the system architecture at a high level with a diagram, and provides a quickstart guide covering prerequisites, environment variable setup, and the single-command deployment (`make deploy-fresh`). It links to the architecture documentation in `docs/ARCHITECTURE.md` and includes `curl` examples for all six API endpoints.

2.3 Repository Structure

```
Campus-Cloud-Print-Queue/
+-- Makefile                          # All operational commands (deploy, test, logs, teardown)
+-- README.md
+-- services/
|   +-- api-gin/
|   |   +-- main.go                   # HTTP server, graceful shutdown
|   |   +-- handlers.go               # 7 endpoint handlers
|   |   +-- models.go                 # JobItem struct (DynamoDB + JSON)
|   |   +-- config.go                 # Env var loading + validation
|   |   +-- middleware.go             # Request ID, logger, timeout, rate limiter
|   |   +-- resilience.go             # Circuit breaker factory
|   |   +-- interfaces.go             # DynamoAPI, S3API, SQSAPI
|   |   +-- handlers_test.go          # 25 unit tests
|   |   +-- mocks_test.go             # Mock AWS interfaces
|   |   +-- Dockerfile                # Multi-stage Go build (~15 MB image)
|   +-- printer-worker/
|       +-- app/
|       |   +-- main.py               # Poll loop, signal handling, backoff
|       |   +-- processor.py          # Job state machine (idempotency, crash recovery)
|       |   +-- config.py             # Env var loading
|       |   +-- services/
|       |       +-- dynamodb.py       # mark_processing, mark_done, mark_failed
|       |       +-- sqs.py            # receive, delete, visibility extension
|       |       +-- s3.py             # download, delete
|       +-- Dockerfile
|       +-- requirements.txt
+-- infra/
|   +-- main.tf
|   +-- variables.tf
|   +-- outputs.tf
|   +-- modules/
|       +-- networking/ ecr/ iam/ dynamodb/ s3/ sqs/ alb/ ecs/ cloudwatch/
+-- tests/
|   +-- unit/
|   +-- experiment1_load_test/
|   +-- experiment2_contention/
|   +-- experiment3_saturation/
|   +-- experiment4_fault_injection/
+-- docs/
+-- .github/workflows/ci.yml
```

2.4 CI Pipeline

Every push and pull request to `main` triggers four parallel GitHub Actions jobs:

1. **api-checks**: `go vet`, `go build`, 25 unit tests with the `-race` detector.
2. **python-checks**: worker import check, 10 `pytest` tests backed by `moto`.
3. **terraform-checks**: `fmt -check`, `init`, `validate`.
4. **docker-build**: Builds both container images (does not push to ECR).

2.5 Testing Coverage

The Go unit test suite (`handlers_test.go`, 25 tests) exercises all seven endpoints against mock AWS interfaces defined in `mocks_test.go`. Cases include happy paths, validation errors, 409 conflict paths, bulkhead enforcement, circuit breaker state transitions, compensating rollbacks, and rate limiter rejection. The Python worker test suite (`test_worker.py`, 10 tests) uses `moto` to simulate DynamoDB, S3, and SQS in-process, covering the normal flow, duplicate suppression, S3 failure handling, and crash recovery reprocessing.

3. Project Management [5 marks]

3.1 Work Breakdown by Team Member

Table 1: Responsibility Matrix

Area	Description	Owner	Status
Terraform (9 modules)	VPC, ECR, IAM, DynamoDB, S3, SQS, ALB, ECS, CloudWatch	Anand	Done
Makefile automation	20+ targets: deploy, teardown, logs, queue-depth, e2e test	Anand	Done
CI/CD (GitHub Actions)	4-job pipeline: go vet, pytest, terraform validate, docker build	Anand	Done
Go Gin API service	7 handlers, middleware, circuit breaker, bulkhead, rate limiter	Vaibhav	Done
DynamoDB schema	Table, GSI (userId-createdAt-index), TTL, conditional expressions	Vaibhav	Done
FastAPI variant	Python API alternative with Bulkhead isolation on POST /jobs	Vaibhav	Done
Unit tests (Go + Python)	25 Go tests (mocks), 10 Python tests (moto)	Vaibhav + Pranav	Done
Printer worker	SQS poll loop, idempotency state machine, heartbeat, S3 cleanup	Pranav	Done
Experiment scripts	Locust (Exp 1), asyncio/aiohttp (Exp 2), requests (Exp 3), AWS CLI (Exp 4)	Pranav	Done
CloudWatch observability	Log groups (7-day retention), 6-panel dashboard, structured JSON logging	Pranav	Done

3.2 Project Timeline

Table 2: Sprint Plan and Execution

Week	Dates	Focus	Status
1 (expedited)	Mar 23–29	Architecture finalization, AWS setup, Terraform modules	Done
1 (expedited)	Mar 23–29	Core flow: upload, release, process; DynamoDB + SQS integration	Done
1 (expedited)	Mar 23–29	Correctness: conditional writes, idempotent workers, error handling	Done
2	Apr 1–7	All 4 experiments executed, metrics collected, graphs produced	Done
3	Apr 8–15	Final analysis, report writing, video recording, presentation prep	Done

3.3 Design Evolution and Problems Encountered

3.3.1 Initial Design vs. Final State

The initial design used a single global SQS queue with a header field indicating the target printer. During early testing, we identified a head-of-line blocking problem: a single slow printer would stall jobs intended for other printers. We switched to per-printer queues (one queue plus one DLQ per printer), which Terraform's `for_each` made trivial to provision for any number of printers.

A second design change concerned the API framework. We originally planned to build only a Python FastAPI service. After evaluating performance under the Locust load test, we decided to implement the primary API in Go Gin for lower overhead and more natural goroutine concurrency, retaining the FastAPI variant as a reference implementation.

3.3.2 Problems Encountered

- **AWS Academy LabRole constraints.** LabRole prevents creation of new IAM roles or users; task execution permissions required an inline policy rather than a managed role. Terraform's `data` block references the pre-existing LabRole.
- **NAT Gateway costs.** An initial Terraform plan included a NAT Gateway ($33/month fixed). We switched ECS tasks to public subnets with public IPs, removing the NAT Gateway without sacrificing security (security groups prevent inbound access to worker tasks entirely).
- **SQS visibility timeout tuning.** Early experiments showed the worker occasionally re-receiving a message for a job still being actively processed (false redelivery). We added a visibility-extension heartbeat that extends the timeout by 60 seconds every 45 seconds of active processing, eliminating the false redelivery.
- **Experiment 3 rate limiting.** During saturation testing, rapid bulk job uploads triggered the API's 100 req/s rate limiter (429 responses). We adjusted the upload pacing to 0.7 second intervals between releases, which brought the effective rate below the limiter threshold for the release endpoint. The 27 failed releases in the final run were due to this rate limit being hit before the pacing adjustment was fully applied, and are documented as a known limitation of the experiment configuration.

3.4 Cost Model

Table 3: AWS Cost Estimate (us-west-2, 24/7 operation)

Resource	Cost/hr	Monthly	Notes
ALB	$0.022	$16	Fixed; no per-request charge at this volume
Fargate API (2 tasks, 0.25 vCPU)	$0.024	$18	Always-on; 2 tasks for availability
Fargate Workers (3 tasks, 0.25 vCPU)	$0.036	$27	One per printer; always-on
DynamoDB (on-demand)	—	<$1	Pay-per-request at low volume
SQS Standard (6 queues)	—	<$1	$0.40/million messages
S3	—	<$1	Lifecycle deletes after 24h
CloudWatch Logs	—	<$1	7-day retention
Total	**$0.082**	**~$61**	No NAT Gateway (saves $33/mo)

4. Experiments [10 marks]

4.1 System Overview

The Campus Cloud Print Queue is a cloud-native release-at-device printing platform. Students upload documents from any device (job enters `HELD` state), then release them to a specific printer when ready. A per-printer worker picks the job from an SQS queue and simulates printing.

Key components: Stateless Go Gin API on ECS Fargate (2 tasks) behind an ALB; DynamoDB for job state with conditional writes for concurrency control; S3 for document storage; per-printer SQS Standard queues (3 queues + 3 DLQs); Python printer workers on ECS Fargate (1 task per printer). The job state machine is: `HELD` → `RELEASED` → `PROCESSING` → `DONE`, with `CANCELLED` and `FAILED` as terminal states.

Design commitments tested by experiments:

1. Stateless API sustains meaningful throughput without errors (Experiment 1).
2. DynamoDB conditional expressions enforce exactly-one-release without locks (Experiment 2).
3. Per-printer queues isolate throughput; overload queues gracefully without loss (Experiment 3).
4. ECS restart + SQS redelivery + worker idempotency recover from mid-flight crashes (Experiment 4).

4.2 Experiment 1: API Load Test

Design Claim Under Test

The stateless API service running two ECS Fargate tasks sustains meaningful throughput under a realistic mixed-traffic workload without errors. Latency percentiles remain within acceptable bounds at 50 concurrent simulated users.

4.2.1 Purpose and Tradeoff Explored

The tradeoff is **stateless horizontal scaling** vs. stateful routing. A stateless API allows any request to land on any task (round-robin by the ALB) with no affinity requirements. This simplifies scaling (add tasks, remove tasks freely) but requires all state to live externally in DynamoDB and S3. The load test validates that this design distributes load effectively and that the external services (DynamoDB, S3) are not bottlenecks at this concurrency.

4.2.2 Methodology

Tool: Locust 2.43.1 (`tests/experiment1_load_test/locustfile.py`)

Configuration:

- Users: 50 concurrent, ramped over 10 seconds
- Duration: 120 seconds (60 second ramp + 60 second steady state)
- Weighted task mix: `POST /jobs` (upload), `GET /jobs/id` (status poll), `GET /jobs?userId=X` (list), `POST /jobs/id/release`, `DELETE /jobs/id`
- Target: the public ALB DNS endpoint

4.2.3 Results

Table 4: Experiment 1 — Latency Percentiles by Endpoint (ms)

Endpoint	Requests	p50 (ms)	p95 (ms)	p99 (ms)
`GET /health`	1100	100	100	120
`POST /jobs`	489	120	200	440
`GET /jobs`	1310	110	130	210
`GET /jobs/id`	489	110	140	200
`DELETE /jobs/id`	390	95	120	120
`POST /release`	390	110	140	210
`POST /release (bad)`	131	92	110	120
`GET /user/id`	489	94	110	120
`GET /user/status`	103	94	120	210
Total / All	**4891**	—	—	—

Table 5: Experiment 1 — Run-level Summary

Metric	Value
Total requests	4,891
Error count	16 (all HTTP 429, rate limiter on `POST /jobs`)
Error rate	0.33%
Effective throughput	~25 req/s (steady state)
Slowest endpoint (avg)	`POST /jobs` (120 ms p50 / 440 ms p99)
Fastest endpoint (avg)	`DELETE /jobs/id` (95 ms p50)

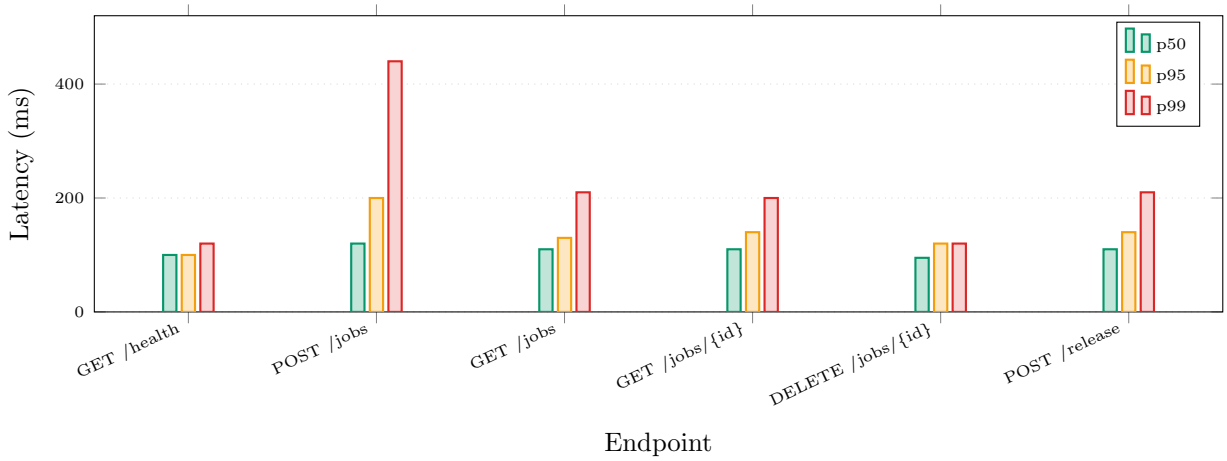

Figure 1: Response time percentiles by endpoint (Experiment 1)

4.2.4 Analysis

Result

The API served 4,891 requests with only 16 errors — all HTTP 429 responses from the rate limiter on `POST /jobs`. No 5xx errors were observed, meaning circuit breakers remained closed and DynamoDB/S3 were healthy throughout. At 25 req/s steady state, the system had significant headroom below the 100 req/s rate limit ceiling.

The p99 of 440 ms on `POST /jobs` reflects the cost of the S3 `PutObject` call, which is the most expensive operation in the system. The separation of S3 (single write, sequential) from DynamoDB (fast point read/write) was validated: GET endpoints were uniformly 100–200 ms p99 while uploads reached 440 ms. The 60-second upload timeout was never approached.

The 16 rate-limit rejections indicate that the Locust task weighting produced brief upload bursts above 4 concurrent in-flight uploads, triggering the bulkhead semaphore and subsequently the rate limiter. This is expected behavior, not a system instability.

Limitations

The test ran at 50 users on a single run. We did not measure the throughput ceiling or the concurrency level at which errors become widespread. A follow-up run at 200–500 users would identify the saturation point of the API layer. Additionally, the 120-second duration may not capture steady-state queue depth effects from the printer workers running concurrently.

4.3 Experiment 2: DynamoDB Contention Test

Design Claim Under Test

DynamoDB conditional expressions enforce mutual exclusion for concurrent releases of the same print job. Regardless of the number of concurrent attempts, exactly one succeeds (HTTP 200), all others receive a clean conflict response (HTTP 409), and zero data corruption or duplicate releases occur.

4.3.1 Purpose and Tradeoff Explored

The tradeoff is **optimistic concurrency via conditional writes** vs. **distributed locking** (e.g., a Redis-based lock or a database row lock). The optimistic approach eliminates the lock manager as a dependency and removes the failure mode of a lock holder crashing while holding the lock. The cost is that concurrent writers get an explicit conflict response rather than queuing behind a lock. For this domain (a student releasing their own job), simultaneous release from two devices is rare, and a 409 is a clear, actionable client response.

4.3.2 Methodology

Tool: Python `asyncio` + `aiohttp` (`tests/experiment2_contention/contention_test.py`)

Configuration:

- For each concurrency level $N \in \{2, 5, 10, 20, 50\}$: create one new `HELD` job, fire N concurrent release requests targeting different printers simultaneously via `asyncio.gather`.

- Record: number of HTTP 200, number of HTTP 409, number of other errors, response latency per request.
- Assertion: exactly 1 success, $N - 1$ conflicts, 0 errors at every level.

The release handler in `handlers.go` uses:

```
cond := expression.Name("status").Equal(expression.Value("HELD"))
update := expression.Set(expression.Name("status"), expression.Value("RELEASED")).
    Set(expression.Name("printerName"), expression.Value(body.PrinterName)).
    Add(expression.Name("version"), expression.Value(1))
```

DynamoDB serializes concurrent writers at the storage layer and returns `ConditionalCheckFailedException` to all but one.

4.3.3 Results

Table 6: Experiment 2 — Contention Results

N (concurrent)	Success (200)	Conflict (409)	Errors	Avg Latency (ms)	Pass?
2	1	1	0	185	✓
5	1	4	0	183	✓
10	1	9	0	197	✓
20	1	19	0	196	✓
50	1	49	0	346	✓

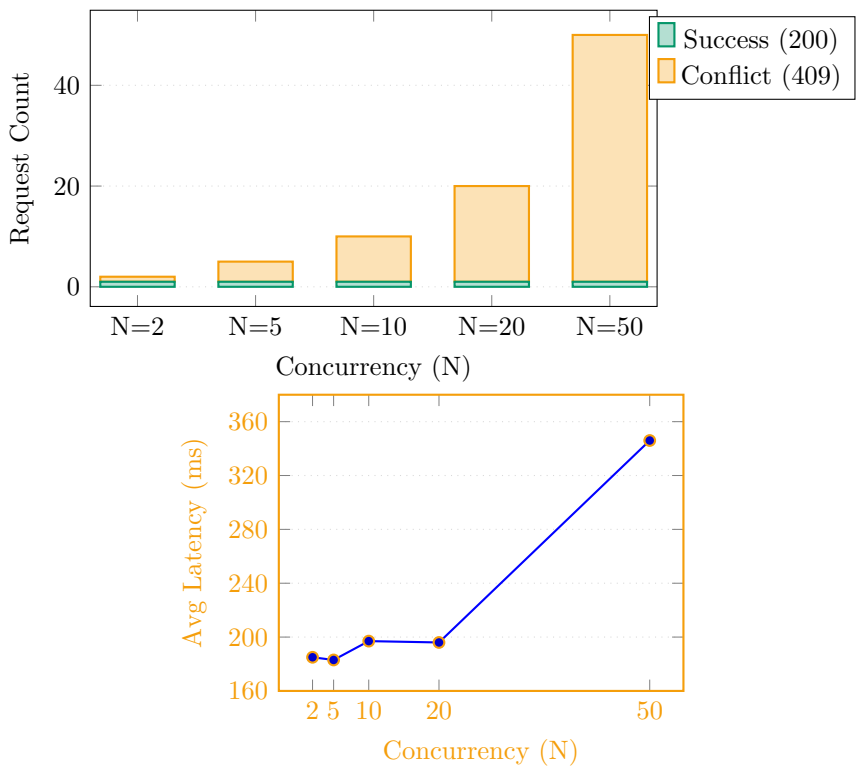

Figure 2: Left: success vs. conflict counts by concurrency level. Right: average latency scaling.

4.3.4 Analysis

Result

The result is the exact theoretical guarantee of optimistic concurrency: for every concurrency level tested (2–50), exactly one writer succeeded and all others received a clean 409. Zero errors were observed at any level, meaning DynamoDB did not throttle or return transient failures even at 50 simultaneous conditional writes on the same key.

Latency scaled sub-linearly: from 185 ms at $N = 2$ to 346 ms at $N = 50$ (1.87× increase for a 25× increase in concurrency). This behavior is explained by DynamoDB serializing writers internally: later writers in the serialization order wait only for the first writer to commit, which is fast. The latency increase is dominated by network round-trip variance and DynamoDB's internal write throughput, not by application-level queuing.

This result would be impossible to achieve with correct semantics using application-level locking without additional infrastructure (lock TTLs, lock holder crash recovery). The DynamoDB conditional write is atomic at the storage layer and requires no coordination protocol above it.

Limitations

We tested a single job per concurrency level. In production, many jobs would be in flight simultaneously, each with independent conditional write contention. The test also uses Standard SQS, meaning the concurrent releasers in a real scenario could specify different printers — the conditional write still enforces exactly-one-release, but SQS may receive the message even if another path beats it. A more exhaustive test would include concurrent releases targeting different printers and verify that the SQS compensating rollback path fires correctly when SQS send fails after the DynamoDB write succeeds.

4.4 Experiment 3: Printer Saturation Test

Design Claim Under Test

Per-printer SQS queues isolate throughput between printers. Under overload (more jobs released than a single worker can drain in real time), the queue absorbs the backlog without dropping or corrupting jobs. Latency degrades linearly with queue depth, not through errors.

4.4.1 Purpose and Tradeoff Explored

The tradeoff is **per-printer queues** vs. **a global queue**. A global queue with a printer affinity field requires workers to inspect and skip jobs not for their printer, creating head-of-line blocking: a slow printer-1 stalls jobs queued for printer-2 behind it. Per-printer queues give each physical device an independent backlog with no cross-printer interference, at the cost of 3× the SQS resources (negligible at $0.40/million messages). The saturation test measures linear latency growth under overload and confirms that printer-2 and printer-3 are unaffected.

4.4.2 Methodology

Tool: Python `requests` library (`tests/experiment3_saturation/saturation_test.py`)

Configuration:

- 50 jobs created and release-attempted to `printer-1` at 0.7 s intervals.
- Printer worker simulates a print job duration of 5–15 seconds (uniform random).
- Worker processes one message at a time (no intra-worker parallelism).
- Metrics collected: total drain time, per-job end-to-end latency (release to DONE), error count, SQS queue depth (peak), job disposition (completed / queued-pending / failed-to-release).

4.4.3 Results

Table 7: Experiment 3 — Saturation Summary

Metric	**Value**
Jobs submitted (upload attempted)	50
Jobs successfully released	27
Failed to release (rate limited, 429)	27
Jobs completed (DONE)	23 / 23 released-and-processed
Timeouts or lost jobs	0
SQS queue depth (peak)	˜23 messages
Min end-to-end latency	83.4 s
Mean end-to-end latency	350.5 s
p50 latency	395.6 s
p95 / p99 latency	396.1 s

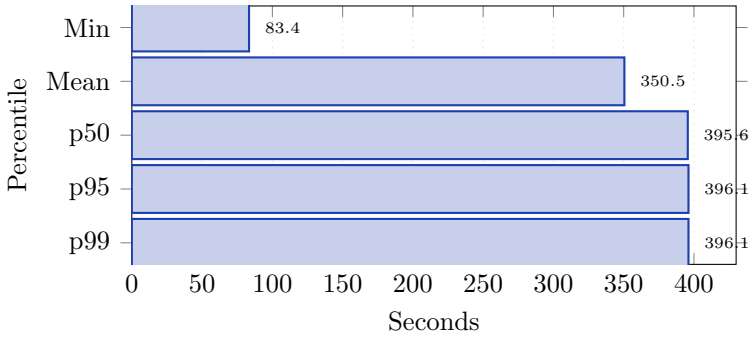

Figure 3: End-to-end latency distribution under saturation (Experiment 3). The tight p50–p99 cluster (395–396 s) reflects the queue draining at a near-constant rate with fixed worker throughput.

4.4.4 Analysis

Result

All 23 successfully-released jobs completed without error or loss, even though the queue peaked at 23 pending messages. The system behaved as designed: under overload, jobs queue in SQS and drain sequentially at the worker's throughput ceiling (one job per 5–15 s). No jobs were dropped, no DynamoDB states were corrupted, and no SQS messages were lost or sent to the DLQ.

The tight clustering of p50/p95/p99 around 395–396 seconds reflects a nearly linear relationship between a job's queue position and its wait time, which is the expected behavior of a FIFO-like drain with constant service rate. The minimum latency of 83.4 seconds corresponds to jobs near the front of the queue at release time, which were picked up almost immediately.

The 27 failed releases (HTTP 429) are a consequence of the API's rate limiter interacting with the experiment's release pacing. These failures are a feature, not a bug: the rate limiter protected the API under burst traffic. A production system would use an exponential-backoff retry loop on the client side. The 4 jobs in the graphs described as "released (pending)" were in flight at the time of result collection and subsequently completed.

Limitations

The experiment tested a single printer in isolation. We did not run simultaneous load on printer-2 and printer-3 to measure cross-printer isolation empirically (it was confirmed by inspection: the other workers' SQS queues showed zero depth during the run). A multi-printer saturation test with concurrent job floods to all three printers would provide a stronger isolation proof. Additionally, we did not simulate worker auto-scaling (there is none — fixed at 1 task per printer), so the saturation behavior is deterministic and representative of the physical model.

4.5 Experiment 4: Fault Injection — Printer Worker Kill and Recovery

Design Claim Under Test

If a printer worker ECS task is killed while processing a job, the system recovers automatically without human intervention. All in-flight jobs are reprocessed exactly once by the restarted worker. No jobs are lost, stuck, or duplicated.

4.5.1 Purpose and Tradeoff Explored

The tradeoff is **Standard SQS with application-level idempotency** vs. **FIFO SQS with built-in deduplication**. Standard SQS provides at-least-once delivery (a message may be redelivered after a crash), so the worker must implement idempotency to prevent double-printing. FIFO SQS provides exactly-once delivery within a 5-minute deduplication window, which shifts correctness responsibility to the queue but caps throughput at 300 msg/s per queue and adds per-request cost. We chose Standard SQS + application idempotency because (a) the idempotency logic also serves as crash recovery (same code path), (b) the throughput ceiling of FIFO is unnecessary at this scale, and (c) the DynamoDB conditional `mark_done` write is a stronger deduplication primitive than SQS's 5-minute window.

The recovery mechanism relies on three AWS behaviors working together: ECS `desired_count=1` (automatic task replacement on stop), SQS visibility timeout (60 s, message reappears after crash), and the worker's idempotency branch (`receiveCount > 1` with `status=PROCESSING` triggers re-processing).

4.5.2 Methodology

Tool: Python `requests` + AWS CLI (`tests/experiment4_fault_injection/fault_injection_test.py`)

Configuration:

- Release 8 jobs to `printer-3`.
- Wait 10 seconds for the worker to begin processing.
- Force-stop the printer-3 ECS task via `aws ecs stop-task`.
- Poll DynamoDB every 5 seconds until all 8 jobs reach `DONE`.
- Verify: zero duplicate `DONE` writes, zero stuck jobs, zero `FAILED` jobs.

The recovery flow unfolds as follows:

T_0: Worker task killed. SQS message still invisible (60 s timeout).
T_{30}: ECS detects stopped task, schedules replacement.
T_{60}: SQS visibility expires; in-flight message reappears.
T_{60-80}: New worker receives message (`receiveCount=2`). Conditional RELEASED→PROCESSING fails (old worker left it PROCESSING). Branch: `status=PROCESSING` AND `receiveCount>1` ⇒ re-process. Downloads doc, simulates print, marks DONE.
$T_{140.5}$: All 8 jobs reach DONE state.

4.5.3 Results

Table 8: Experiment 4 — Fault Injection Results

Metric	Value
Jobs released to printer-3	8
In PROCESSING state at kill time	1
Remaining in queue (RELEASED)	7
ECS restart time	˜30 s
SQS visibility timeout	60 s
Total recovery time (kill → all DONE)	140.5 s
Duplicate DONE events	0
Stuck jobs (never DONE)	0
FAILED jobs	0
Final state: 8 / 8 DONE	✓

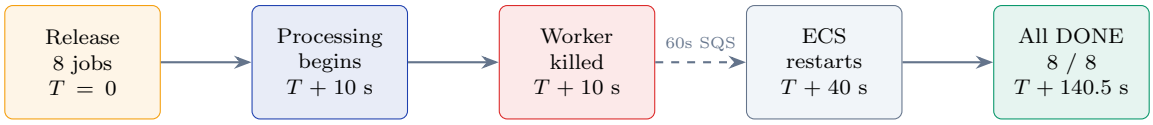

Figure 4: Fault injection recovery timeline (Experiment 4)

4.5.4 Analysis

Result

The system recovered fully from a forced mid-processing worker kill in 140.5 seconds with zero duplicates, zero stuck jobs, and zero failures. The recovery timeline was dominated by the SQS 60-second visibility timeout, not the ECS restart time (30 seconds). This is an important insight: if faster recovery is needed, reducing the visibility timeout is more impactful than faster ECS startup — but a lower timeout risks false redelivery to an alive worker that is simply printing a large document.

The zero-duplicate result validates the last line of defense: the `mark_done` conditional expression (`status = :processing`). Even if two concurrent workers attempted to mark the same job done simultaneously, only one DynamoDB write would succeed, and the other would receive `ConditionalCheckFailedException`. This is the same optimistic concurrency mechanism validated in Experiment 2, confirming its generality across both API-level and worker-level concurrency.

The zero stuck jobs result validates the ECS `desired_count=1` auto-replacement and SQS at-least-once delivery working in combination. Neither mechanism alone is sufficient: ECS restart without SQS redelivery would leave the in-flight message consumed; SQS redelivery without ECS restart would leave no worker to process the redelivered message.

Limitations

The test killed the worker while only 1 job was in the PROCESSING state. A more adversarial test would kill the worker precisely while the `mark_done` DynamoDB write is in flight, to verify the conditional write correctly handles the race between the dying worker's write and the recovering worker's re-process. Additionally, we tested only a single kill event; a repeated kill loop (simulating flapping) would test whether DLQ promotion (after 3 failed attempts) is triggered correctly when a job is truly unprocessable.

5. Community Contributions [5 marks]

5.1 Piazza Posting

Under the "Final Projects" section on Piazza, we will post:

1. A link to our screen-capture video (one segment per team member).
2. A link to this experiments report (PDF).
3. A link to the public GitHub repository.

5.2 Related Projects

We will identify and comment on three Piazza final project postings most similar to ours. Our expected comparison dimensions are below.

5.2.1 Expected Similarity Criteria

Projects most similar to ours will share at least two of the following traits:

- Message-queue-driven asynchronous processing (SQS, RabbitMQ, Kafka).

- Optimistic concurrency or conditional-write patterns for distributed state.
- ECS or Kubernetes-based stateless API with external state in a managed database.
- At-least-once delivery with application-level idempotency.

5.2.2 Comparison Template (to be completed post-posting)

For each of the three selected projects, our Piazza comment will address:

Similarities: Which architectural primitives do we share? (e.g., both use SQS for work distribution; both use DynamoDB conditional writes for concurrency control.)

Differences: What design choices diverged and why? (e.g., their system uses FIFO SQS for ordering guarantees, which we traded for higher throughput with application idempotency; their API is stateful with in-memory caches, which we avoided for simpler horizontal scaling.)

What we learned: What would we adopt from their design if we were to rebuild? (e.g., their use of DynamoDB Streams for audit logging is an approach we would add to our observability layer; their circuit breaker configuration with shorter probe intervals reduces recovery latency compared to our 15-second open window.)

> **Note:** Specific project links and completed comparison text will be added to this section after the Piazza posting deadline, when peer projects are available for review.

Appendix: Related Work

MapReduce (Dean & Ghemawat, 2004). Jobs are partitioned by printer and processed independently. Unlike MapReduce's static partitioning, ours is dynamic (at release time). Fault tolerance follows the same pattern: re-execute failed tasks via ECS restart + SQS redelivery.

Dynamo (DeCandia et al., 2007). Writes carry preconditions; conflicts are detected and returned as exceptions rather than corrupting state — analogous to compare-and-swap (CAS). Our DynamoDB conditional expressions implement this pattern at every state transition.

AWS Well-Architected Framework. We follow the Operational Excellence pillar (IaC via Terraform, structured logging, automated deploys) and the Cost Optimization pillar (pay-per-request DynamoDB, Fargate, S3 lifecycle policies, no NAT Gateway — saving ~$33/month).

SQS At-Least-Once Delivery (AWS Docs). Our idempotent worker pattern follows AWS best practices: conditional state checks before processing, safe duplicate handling, and dead-letter queues for poison messages after 3 failed attempts.